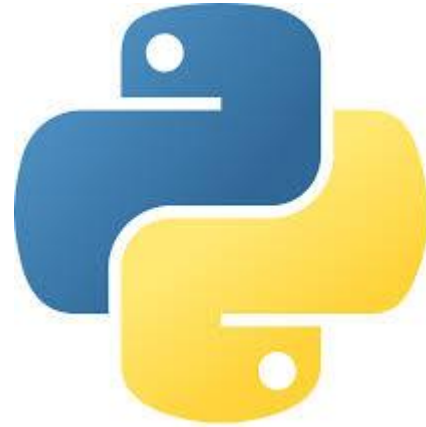




Python Programming

Shaik Abdul Gaffar, MCA
Business & Management Department
Yogi Vemana University



**What is a programming
language?**

What is a programming language?

- a set of instructions that you give to a computer so that it will do a particular task
- A programming language is a way to give instructions to a computer so it knows what to do and how to do it.



Generations of Programming Languages

- Programming languages have evolved over time.
- They are classified into five generations (1GL to 5GL).
- Each generation made programming easier and more powerful.



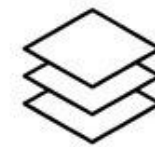
Each Generation Improves



Ease of Use



Efficiency



Abstraction

1GL – Machine Language

- Written in 0s and 1s
- Directly understood by computer
- Very fast but difficult for humans
- Machine dependent

Example: **10101010**

2GL – Assembly Language

- Uses mnemonics instead of binary
- Easier than machine language
- Requires an Assembler
- Machine dependent

Examples: **MOV, ADD, SUB**

3GL – High-Level Languages

- Close to human language
- Easy to learn, write; and debug
- Machine independent
- Requires Compiler or Interpreter

Examples: **C, C++, Java, Python**

4GL – Very High-Level Languages

- Less coding, more automation
- Focus on what to do, not how
- Mainly used in databases & data analysis

Examples: **SQL, MATLAB, R, SAS**

5GL – AI & Logic-Based Languages

- Based on logic and rules
- Used in Artificial Intelligence
- Programmer defines the problem, system finds solution

Examples: **Prolog, Mercury**

Special Programming Languages

Special programming languages are used for specific tasks rather than general-purpose programming. They include:

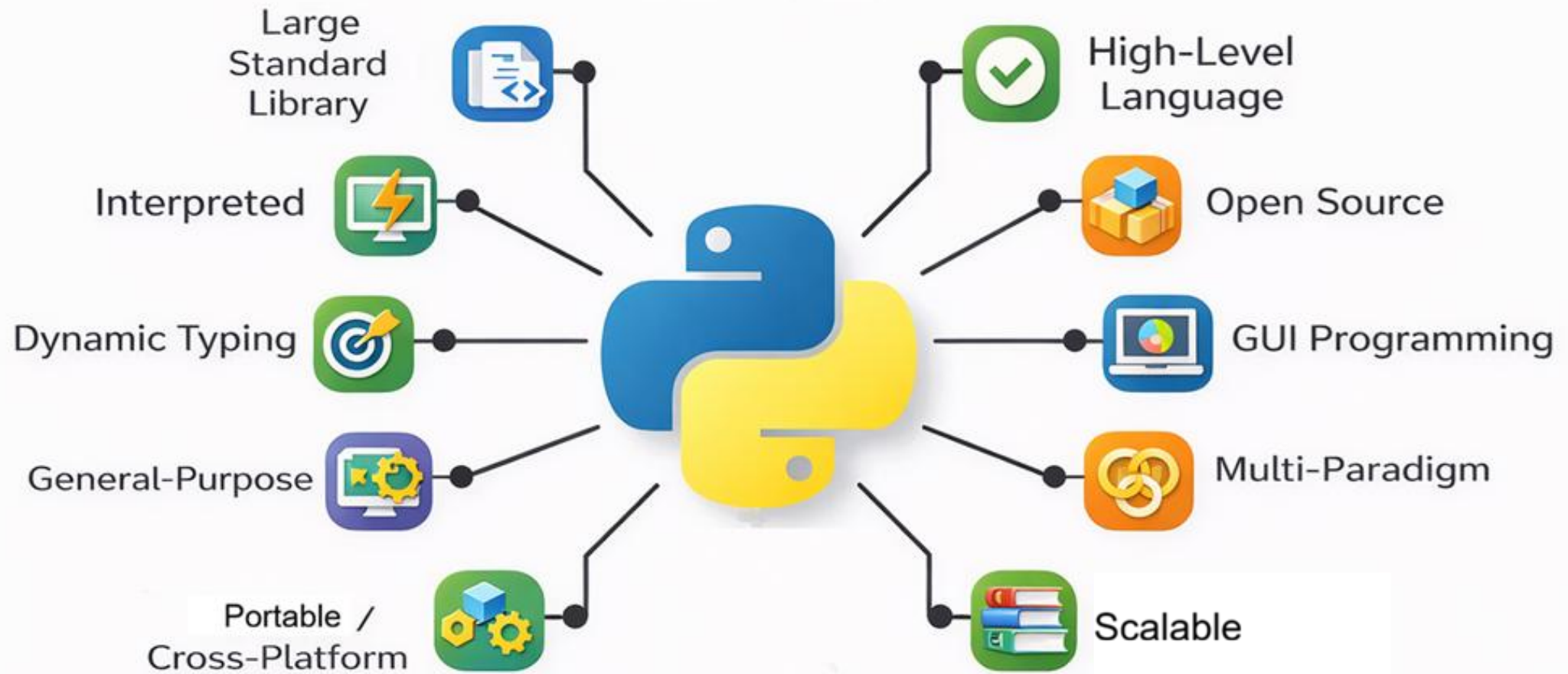
- Scripting Languages: **JavaScript, VBScript, PHP, Python, Perl, Ruby**
- Command Languages: **Bash, Sh, Ch**
- Document Formatting Languages: **TeX, PostScript, Latex**
- World Wide Web Display Languages: **HTML, CSS, XML**

What is Python Programming Language ?

- Python is a general-purpose, dynamic, high-level, interpreted, object-oriented programming language that is easy to learn and widely used in modern technologies.
- Python was developed by **Guido van Rossum**
- Development started in 1989 and first released in 1991
- Developed at Centrum Wiskunde & Informatica (CWI), Netherlands



Features of Python



Advantages & Disadvantages of Python

Advantages

- Easy to Learn and Use
- Versatile and Flexible
- Improved Productivity
- Highly Secure
- Strong Community Support



Disadvantages

- Slow Speed
- Memory Inefficient
- Weak Mobile Computation
- Poor Database Access
- Runtime Error

Background & Need for Python

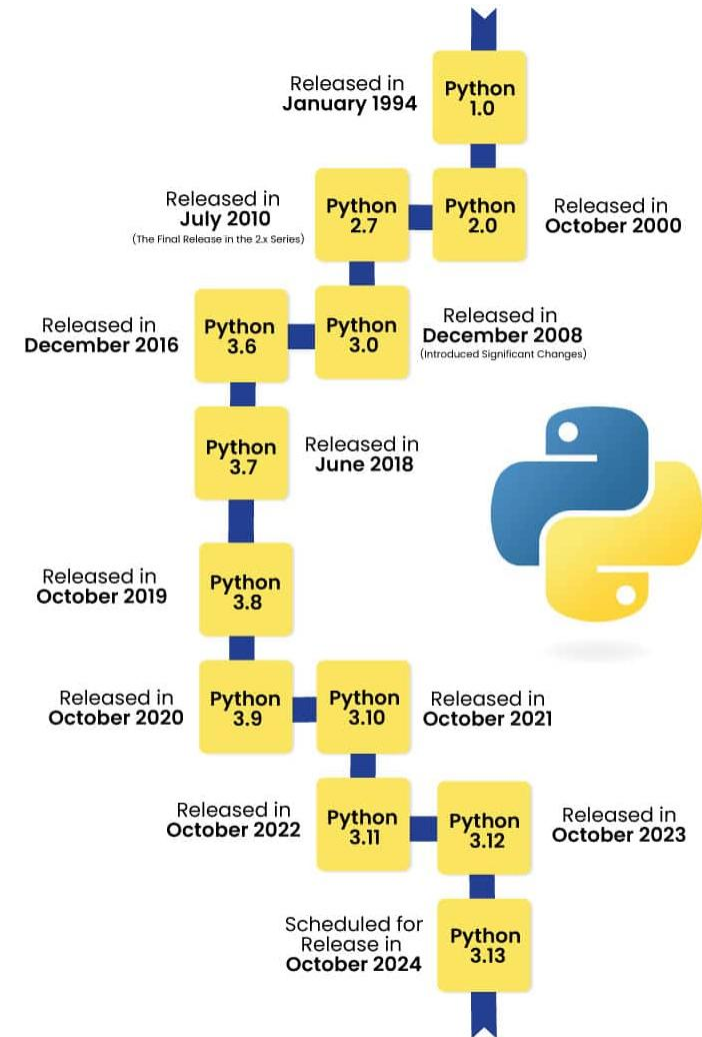
- In the late 1980s and early 1990s, most popular programming languages had clear drawbacks:
 - **C / C++** → Very powerful but complex syntax, memory management issues
 - **Perl** → Powerful for scripting but difficult to read and maintain
- There was a strong need for a language that was:
 - Simple and readable
 - Powerful like C
 - Flexible like scripting languages
 - Suitable for both small scripts and large applications
- Python was designed to bridge the gap between system-level languages (C, C++) and scripting languages (Perl, Shell).

Influence of Other Programming Languages

- Python did not come from scratch. It borrowed best ideas from multiple languages:
 - **ABC Language:** Simple syntax, Indentation-based structure (instead of braces {})
 - **C Language:** Core design philosophy, Ability to extend Python using C modules
 - **Modula-3:** Exception handling, Module system
 - **Perl:** Powerful string handling, Scripting capabilities
 - **Lisp:** Functional programming concepts, Dynamic typing ideas

Major Versions of Python

- Python has evolved through multiple versions
- Each major version introduced new features and improvements
- The versions are mainly classified as:
 - Python 1.x (1991)
 - Python 2.x (2000)
 - Python 3.x (2008 - Present)



Python 1.x (1991)

- First official release of the Python programming language
- Focused on simplicity, readability, and ease of use
- Laid the foundation for future Python versions

Key Features

- Supported core data types: Numbers, strings, and lists
- Provided: Functions and modular programming, Basic exception handling
- Suitable for: Small programs, Scripting tasks
- Mainly used in: Academic environments, Research-based applications
- Had limited library support compared to later versions

Python 2.x (2000)

- Released to improve performance and usability over Python 1.x
- Became the most widely adopted version of Python for many years
- Extensively used in:
 - Industry applications
 - Web development
 - System scripting

Key Improvements

- Introduced important features:
 - List comprehensions
 - Automatic garbage collection
 - Limited Unicode support
- Faster execution compared to Python 1.x
- Supported a large number of third-party libraries

End of Python 2.x & Language Changes

- Python 2.7 was the final release in the 2.x series
- Backward compatibility:
 - Slowed innovation and modernization
- Official support ended in January 2020
- Developers were encouraged to migrate to Python 3

Syntax Differences (Python 2 → Python 3)

- **Input handling:**
 - Python 2: `raw_input()` and `input()`
 - Python 3: only `input()`
- **Print statement:**
 - Python 2: `print "Hello"`
 - Python 3: `print("Hello")` (function)

Python 3.x (2008 – Present)

- Designed to overcome the limitations of Python 2
- Focused on:
 - Long-term improvement
 - Code clarity and consistency
- Not backward compatible with Python 2
- Represents a major redesign of the Python language

Key Improvements

- Unicode support by default
- Improved standard libraries
- Better performance and stronger security
- Suitable for modern and large-scale applications

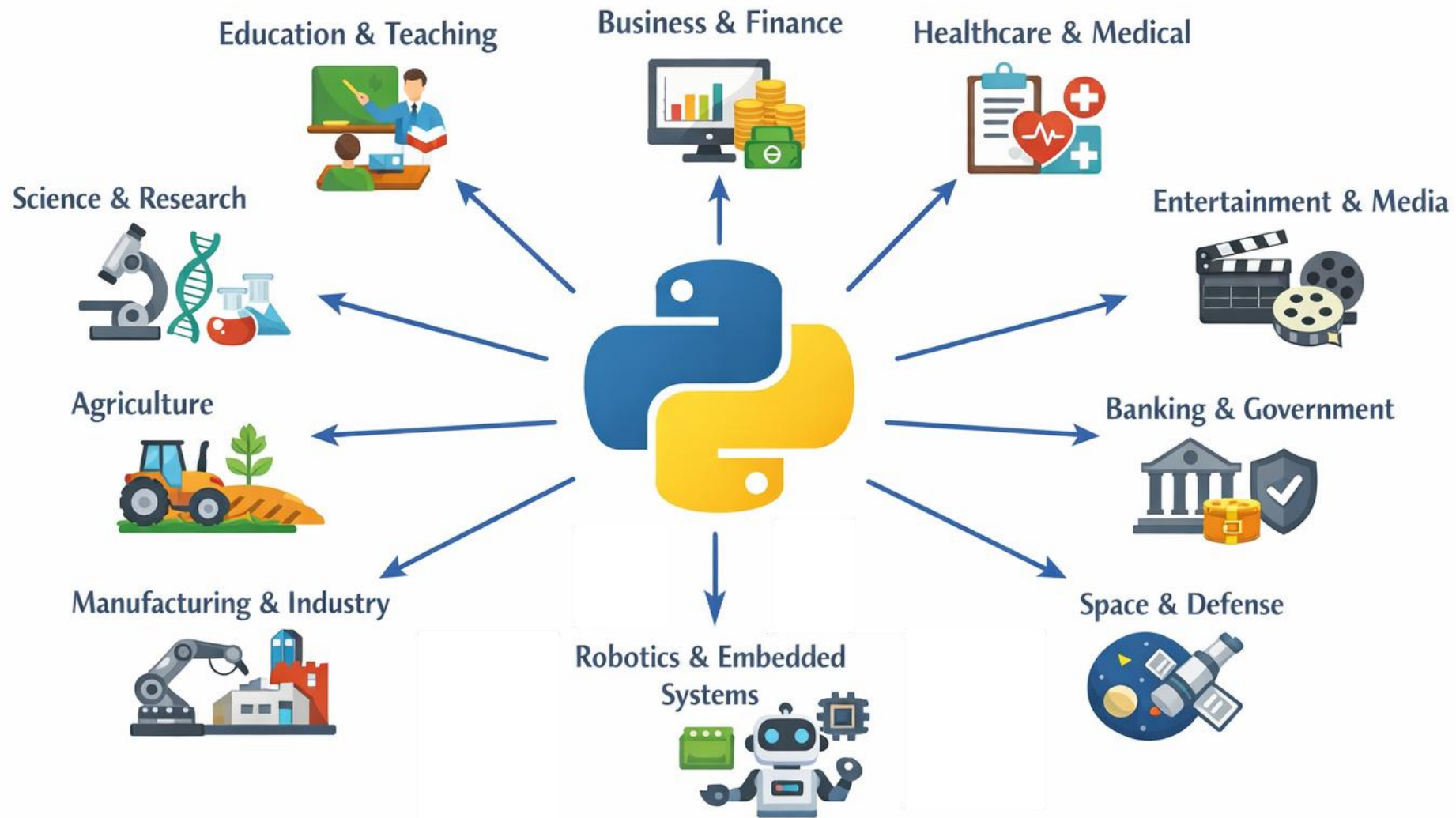
Major Python 3 Versions & Enhancements

- **Python 3.6:** Introduced f-strings for easy string formatting
- **Python 3.8:** Introduced Walrus operator (`:=`) for assignment in expressions
- **Python 3.9:** Added dictionary merge and update operators
- **Python 3.10:** Introduced Structural Pattern Matching
- **Python 3.11:** Major performance improvements
- **Python 3.12+:** Cleaner internal architecture, Further speed and efficiency enhancements

Python 3 Present & Future

- All new features are released only in Python 3
 - Widely used in:
 - Web development
 - Data science
 - Artificial Intelligence
 - Automation and cybersecurity
- Considered the official and future version of Python

Applications of Python in Various Fields



Basic Syntax of Python

- **Simple Definition**

“Syntax is the correct way of writing Python code so that Python can understand and execute it.”

- **Why Python Syntax is Special?**

- ✓ Simple
- ✓ English-like
- ✓ Less symbols
- ✓ Easy for beginners and non-tech students

- **Basic Elements of syntax:**

- | | | |
|-------------|---------------|----------------------|
| • Comments | • Data Types | • Escape Sequences |
| • Keywords | • Indentation | • Control Statements |
| • Variables | • Functions | • Operators |

Writing your first Python Program

```
print("Hello, World!")
```

- `print()` → A built-in function
- `"Hello, World!"` → A string (text inside quotes)
- This line tells Python to display text on the screen

Comments in Python

- **Formal Definition**

“A comment in Python is a line of text that is ignored by the Python interpreter and is used to explain the code for better understanding and readability.”

- **Why do we need comments ?**

- The programmer (you)
- Your team members
- You after 6 months

- **Without comments:**

- Code becomes confusing
- Difficult to maintain
- Hard to explain in interviews

- Comments improve program readability and make maintenance easier.

Types of Comments in Python

Single Line Comments

- **Method 1: Single-Line Comment**

- Used to explain code
- Starts with #
- **Syntax:**

This is a single-line comment

- **Example:**

```
# This program prints a message  
print("Hello MBA Students")
```

- **Method 2: Inline Comment**

- Written on the same line as code
- Explains the purpose of that line
- **Example:**

```
marks = 75 # Student marks
```

Multi-Line Comments in Python

- **Method 1: Multiple # Symbols (Recommended)**

- Used for multi-line explanations
- Clear and readable
- **Example:**

```
# This program  
# calculates  
# total marks
```

- **Method 2: Triple Quotes (''' or ''')**

- Mainly used for documentation
- Not a true comment
- **Example:**

```
''''  
This program calculates  
total and average marks  
''''
```

Indentation in Python

- **Formal Definition**

“Indentation in Python refers to the spaces at the beginning of a line of code that define the structure and blocks of a program.”

- It shows grouping
- It shows decision and flow
- It is mandatory in Python
- A code block is a group of statements that run together as one unit.
- In Python, code blocks are created using indentation and are commonly used with: if / else, for, while, def (functions)
- **Why Python Uses Indentation?**
 - To Make Code Readable
 - To Avoid Confusion
 - To Force Discipline

Keywords in Python

- **Simple Definition**

“Keywords are special words in Python that are already defined by the language and are used to perform specific actions.”

- Fixed meaning
- Fixed purpose
- Cannot be changed
- Keywords cannot be used as variable names.

if = 10

Python will throw an error.

SyntaxError: invalid syntax

- **Why Keywords are Important?**

- They tell Python what to do
- They control decision making
- They control loops and flow
- They help define functions, classes, logic

- Without keywords Python cannot understand instructions.

- How Many Keywords are There?

<i>and</i>	<i>continue</i>	<i>finally</i>	<i>is</i>	<i>raise</i>
<i>as</i>	<i>def</i>	<i>for</i>	<i>lambda</i>	<i>return</i>
<i>assert</i>	<i>del</i>	<i>from</i>	<i>nonlocal</i>	<i>True</i>
<i>async</i>	<i>elif</i>	<i>global</i>	<i>None</i>	<i>try</i>
<i>await</i>	<i>else</i>	<i>if</i>	<i>not</i>	<i>while</i>
<i>break</i>	<i>except</i>	<i>import</i>	<i>or</i>	<i>with</i>
<i>class</i>	<i>False</i>	<i>in</i>	<i>pass</i>	<i>yield</i>

Escape Sequence in Python

- **Simple Definition**

“Escape sequences are special codes used inside strings to control formatting like new lines, tabs, quotes, and special symbols.”

- Used inside strings
- Starts with \ (backslash)
- Helps in text formatting

- **Why Do We Need Escape Sequences?**

How will you print this sentence?

He said, "Python is easy"

Most Common Escape Sequences in Python

- **\n New Line** → Moves text to next line

```
print("Hello\nWelcome")
```

- **\t Tab Space** → Adds horizontal space (used in tables)

```
print("Name\tMarks")
```

- **\\" Double Quote** → Prints double quotes inside string

```
print("He said, \"Python is easy\"")
```

- **\' Single Quote** → Prints single quote inside string

```
print('It\'s a good day')
```

- **\\ Backslash** → Prints backslash (important for file paths)

```
print("C:\\Users\\Admin")
```

- **\r Carriage Return** → Moves cursor to line start (overwrites text)

```
print("Hello\rWorld")
```

- **\b Backspace** → Removes one character

```
print("Hello\b")
```

Variables in Python

- **Simple Definition**

“A **variable** is a name given to a value so that we can use and change it later in a program.”

- Stores data
- Has a name
- Value can change

- **How Variables Work?**

- Python creates memory
- Value is stored in memory
- Variable name points to that value

profit = 50000

profit → name

50000 → value

How to Declare a Variable in Python?

Syntax:

```
variable_name = value
```

Example:

```
sales = 100000  
name = "Rahul"  
rating = 4.5
```

- No need to mention data type
- Python decides automatically
- This feature is called Dynamic Typing.

Rules for Naming Variables

1. Must start with letter or underscore _
2. Can contain letters, numbers, _
3. Cannot start with number
4. Cannot be a keyword
5. Case-sensitive (Age ≠ age)

Various variable scenarios

- Variable Value Can Change
- Multiple Assignment in Python
- Same Value to Multiple Variables

Variable Naming Conventions Case

- **snake_case** → student_name, total_marks
Used for variables & functions (Python standard)
- **camelCase** → studentName
Common in Java/JavaScript, avoid in Python
- **PascalCase** → StudentDetails
Used for class names
- **SCREAMING_SNAKE_CASE** → MAX_LIMIT, PI
Used for constants

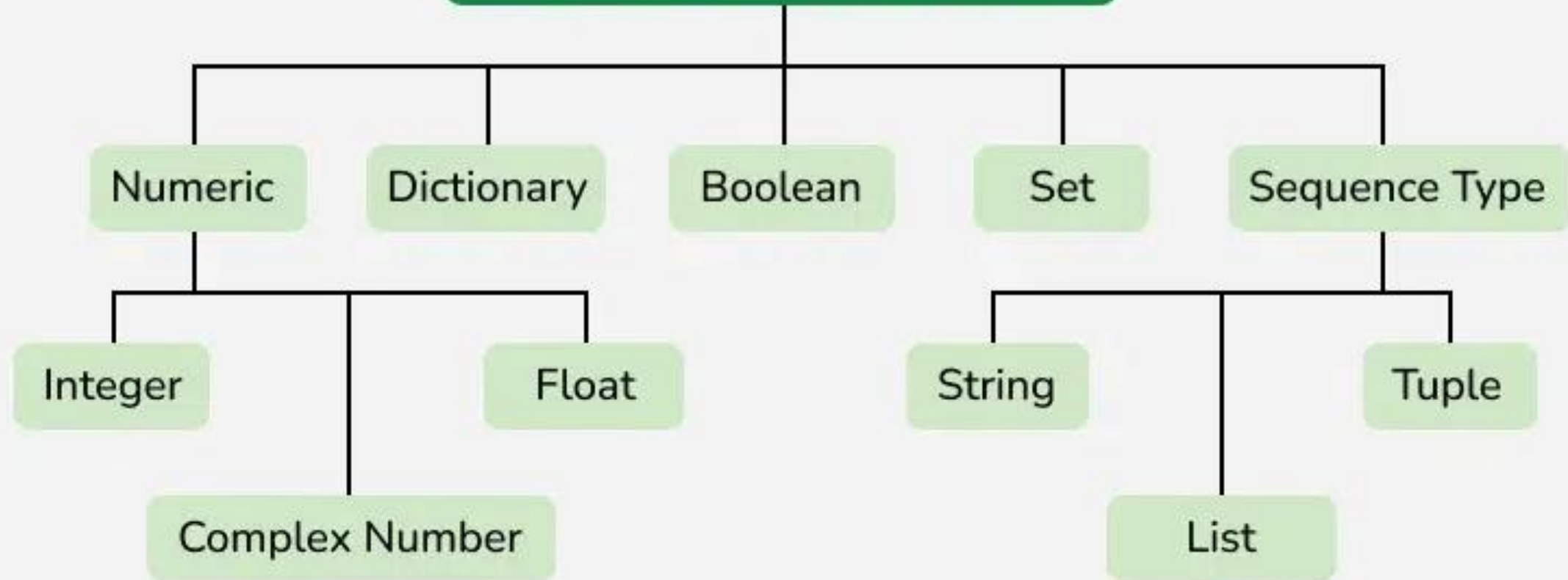
Data Types in Python

- **Formal Definition**

“A data type in Python defines the type of value a variable can hold and determines what operations can be performed on that value.”

- Python automatically understands the data type. This feature is called Dynamic Typing.
- Why Do We Need Data Types?
 - Prevent mistakes
 - Help Python understand operations
 - Improve performance and accuracy

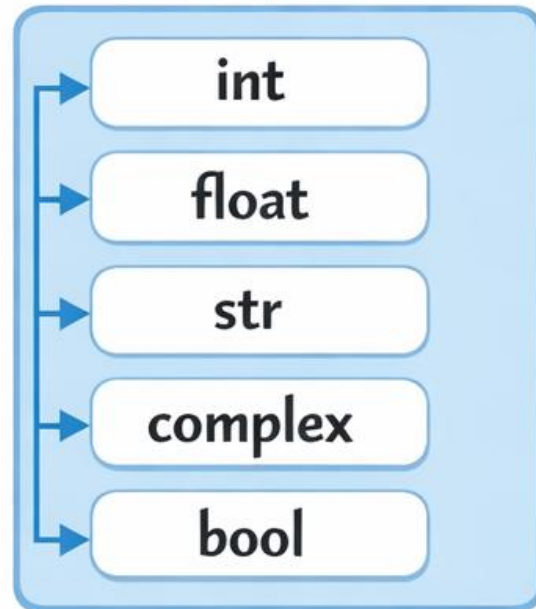
Python Data Types



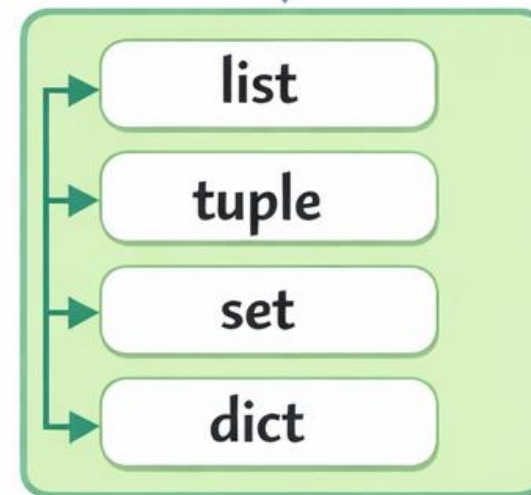
Classification of Data Types in Python

```
graph TD; A[Classification of Data Types in Python] --> B[Basic (Primitive) Data Types]; A --> C[Advanced (Non-Primitive) Data Types]; B --> D[int]; B --> E[float]; B --> F[str]; B --> G[complex]; B --> H[bool]; C --> I[list]; C --> J[tuple]; C --> K[set]; C --> L[dict];
```

Basic (Primitive) Data Types



Advanced (Non-Primitive) Data Types



Basic Data Types

int - Integer Data Type

Stores whole numbers (no decimal)

age = 21
salary = 50000

- ✓ Positive or negative
- ✓ No decimal point

float - Floating Point Data Type

Stores decimal values

price = 99.99
rating = 4.5

- ✓ Profit percentage
- ✓ Interest rate

str - String Data Type

Stores text or characters

"name = "Ravi"
company = "Infosys"

- ✓ Written inside quotes
- ✓ Letters, numbers, symbols

complex - Complex Data Type

Stores numbers in the form:

$x = 2 + 3j$

- ✓ Engineering
- ✓ Scientific Calculations



bool - Boolean Data Type

Stores only **True** or **False**!

True **False**

- ✓ is_active = True
- ✓ Loan Approved?

Advanced Data Types

list – List Data Type

Stores multiple values in a single variable.

Example: marks = [70,80,90]

- ✓ Ordered
- ✓ Changeable
- ✓ Allows duplicates



tuple – Tuple Data Type

Stores multiple values but cannot be changed.

Example: days = ("Mon", "Tue", "Wed")

- ✓ Ordered
- ✓ Not changeable (immutable)



set – Set Data Type

Stores unique values only.

departments = {"HR", "Finance", "IT"}

- ✓ No duplicates
- ✓ Unordered



dict – Dictionary Data Type

Stores data in key : value pairs.

student = {"name": "Abdul", "marks": 85}

- ✓ Employee records
- ✓ Product details

